

# 高级语言程序设计

## 实验报告

南开大学 工科大类

姓名：张毓珊

学号：2512481

班级：工科试验班模拟 2-2

2026 年 5 月 12 日

## 目录

|                      |   |
|----------------------|---|
| 高级语言程序设计大作业实验报告..... | 2 |
| 一. 作业题目.....         | 2 |
| 二. 开发软件.....         | 2 |
| 三. 课题要求.....         | 2 |
| 四. 主要流程.....         | 2 |
| 1. 动画系统封装.....       | 2 |
| 2. 无限地图与相机跟随.....    | 4 |
| 3. UI 与场景切换.....     | 4 |
| 4. 冻结标志阻断主循环.....    | 4 |
| 5. 战斗与交互机制.....      | 5 |
| 6. 内存管理与优化.....      | 5 |
| 五. 收获.....           | 6 |

# 高级语言程序设计大作业实验报告

## 一. 作业题目

基于 Qt 的横版跑酷闯关游戏

## 二. 开发软件

Qt 6.10.2

## 三. 课题要求

- 1) 面向对象。
- 2) 使用 C++
- 3) 实现图形化

## 四. 主要流程

### 1. 动画系统封装

游戏需要支持多种角色动画（待机、行走、跳跃、攻击、眠龙剑大招），每种动画的帧数、每帧持续时间、是否循环各不相同。初期采用统一帧间隔播放，导致跳跃动画不跟手、大招动画播放速度异常，且大招动画会被普通动画瞬间覆盖。原因是固定帧间隔无法适配不同动作的节奏需求；普通动画切换逻辑未考虑大招的高优先级。

解决方式，封装 AnimationClip 类存储帧序列和每帧独立时长，AnimationPlayer 类负责基于 delta 时间的精确播放。通过动作优先级机制，确保大招播放时屏蔽所有普通动画切换。

```

20 void AnimationPlayer::update(float deltaSeconds)
21 {
22     if (!playing || !clip) return;
23
24     const auto& frames = clip->frames();
25     if (frames.empty()) return;
26
27     accumulatorMs += deltaSeconds * 1000.0f;
28     int duration = frames[currentFrame].durationMs;
29     while (accumulatorMs >= duration && playing)
30     {
31         accumulatorMs -= duration;
32
33         int nextFrame = currentFrame + 1;
34         if (nextFrame >= (int)frames.size()) {
35             if (loop) {
36                 nextFrame = 0;
37             } else {
38                 currentFrame = (int)frames.size() - 1;
39                 playing = false;
40                 break;
41             }
42         }
43
44         currentFrame = nextFrame;
45
46         if (playing) duration = frames[currentFrame].durationMs;
47     }
48 }

```

```

//动画
if (!isAttacking && !isPlayingSuper) {
    if (!onGround) {
        if (jumpClip && animPlayer.getCurrentClip() != jumpClip) {
            animPlayer.play(jumpClip, true);
        }
    } else {
        bool moving = (leftPressed || rightPressed);
        if (moving) {
            if (walkClip && animPlayer.getCurrentClip() != walkClip) {
                animPlayer.play(walkClip, true);
            }
        } else {
            if (idleClip && animPlayer.getCurrentClip() != idleClip) {
                animPlayer.play(idleClip, true);
            }
        }
    }
}
}

```

## 2 . 无限地图与相机跟随

初期采用简单背景偏移，出现快速移动时背景黑边等问题。解决方法：采用“相机跟随 + 循环背景拼接”方案实现伪无限地图。每帧根据玩家位置计算相机坐标，绘制时所有世界坐标平移-cameraX，并循环绘制多份背景图确保无黑边。

```
int targetX = player.getCollisionRect().center().x() - 256;
cameraX = targetX;
// 最小值不低于0
if (cameraX < 0) cameraX = 0;

// 相机
bufferPainter.translate(-cameraX, 0);

if (!background.isNull()) {
    int sw = 1024;
    int startX = (cameraX / sw) * sw - sw;

    for (int x = startX; x <= cameraX + 512; x += sw) {
        bufferPainter.drawPixmap(x, 0, background);
    }
} else {
    bufferPainter.fillRect(0, 0, 512, 512, QColor(80, 100, 150));
}
```

## 3 . UI 与场景切换

游戏需要在开始界面、帮助界面、游戏中、胜利、结束五种状态间切换，并实时显示碎片数量和倒计时。使用枚举 gameState 管理场景。paintEvent 根据 gameState 绘制不同界面。碎片数量和倒计时通过 QPainter::drawText 实时绘制在屏幕左上角和中央上方。

碎片数量达到 20 时，额外显示“按 R 释放大招通关”的红色提示。倒计时归零或玩家死亡时切换至结束界面。

## 4 . 冻结标志阻断主循环

在 gameUpdate 开头检查 isFrozen，若为真则只调用 update() 重绘并返回，不执行任何移动、碰撞、刷怪、计时等更新。

```

if (isFrozen) {
    update();
    return;
}

```

## 5 . 战斗与交互机制

攻击判定：遍历敌人，检测玩家攻击矩形与敌人碰撞盒相交，一次攻击只命中第一个敌人。

```

void GameWidget::checkHit(int damage)
{
    QRectF attackRange = player.getAttackRange();

    for (int i = 0; i < enemies.size(); ++i) {
        if (attackRange.intersects(enemies[i]->getCollisionRect())) {
            enemies[i]->takeDamage(damage);
            break;
        }
    }
}

```

受伤无敌：使用 isPlayerInvincible 标志和 QTimer::singleShot(500ms) 实现 0.5 秒无敌时间，防止连续扣血。

```

225   for (int i = 0; i < enemies.size(); i++) {
226       if (!isPlayerInvincible && enemies[i]->getCollisionRect().intersects(player.getCollisionRect())) {
227           player.takeDamage(5);
228           isPlayerInvincible = true;
229           QTimer::singleShot(500, this, &GameWidget::onInvincibilityEnd);
230           break;
231       }
232   }

```

技能冷却：Skill 类存储 cooldownMs（毫秒），cast()时转换为秒，每帧通过 updateCooldown(deltaSec)减少。

二段跳：利用 canDoubleJump 和 doubleJumpUsed 标志，地面跳跃时重置，空中且满足条件时执行二段跳。

## 6 . 内存管理与优化

游戏中动态生成敌人和碎片，需要妥善释放内存。

实现：在 GameWidget 析构函数和 resetToStart 函数中，遍历 enemies 和 fragments 列表，手动 delete 每个指针对象，然后 clear 列表。确保基类 Entity 的析构函数为 virtual，避免内存泄漏。

```

void GameWidget::resetToStart()
{
    gameState = PHASE_START;
    isRunning = false;

    for (int i = 0; i < enemies.size(); i++) {
        delete enemies[i];
    }
    enemies.clear();

    for (int i = 0; i < fragments.size(); i++) {
        delete fragments[i];
    }
    fragments.clear();
    collectedFragments = 0;
    distanceSinceLastFragment = 0.0f;

    gameTimer = 0.0f;
    freezeNpcIndex = 0;
    isFrozen = false;

    spawnTimer = 0.0f;
    spawnInterval = 3.0f;

    player.takeDamage(-200);
    player.setPosition(QPointF(100, 396));
    cameraX = 0; // 重置相机
    repaint();
}

```

## 五. 收获

理解了帧动画的原理与控制方法；掌握了动态内存管理的重要性，学会了如何避免内存泄漏，正确管理动态生成的对象；体会到了游戏手感调优的重要性，通过反复调整参数和逻辑，实现了流畅自然的操作体验；学会了如何将剧情设定与游戏玩法结合，通过眠龙碎片收集和大招系统，打造了完整的游戏闭环。

本次开发也让我认识到了自己的不足，比如在架构设计上还不够清晰，代码功能划分不够明确。未来我会继续学习游戏引擎的设计思想，提升自己的代码架构能力。